

relatorio

June 9, 2026

1 Informações dos Desenvolvedores

Nome: Renato Lima do Nascimento - RM:570266
Nome: Renato Levy do Valle - RM:572352
Nome: Alexandre Martins Niewelt - RM:570614

2 Importação de Bibliotecas

No início do código utilizamos bibliotecas nativas do Python:

```
import csv
import os
```

A biblioteca **csv** é usada para ler o arquivo de telemetria. Como o projeto utilizar um arquivo externo **dados.csv**, o Python precisava de uma forma organizada para percorrer cada linha desse arquivo, já que a decisão foi não utilizar bibliotecas avançadas como **Pandas**.

A biblioteca **os** é usada para lidar com os caminhos de arquivos e pastas. Isso é necessário porque o arquivo **systemas.py** fica dentro da pasta **src** enquanto o CSV fica dentro da pasta **data**.

Essa parte representa a separação entre código e dados. O programa não depende mais de todos os valores escritos diretamente no código. Ele lê os dados de uma forma externa.

3 Nome do Sistema e Caminho do Arquivo

O sistema possui uma constante com o nome do projeto:

```
nome_sistema = "Sistema Inteligente de Monitoramento Espacial - Aurora Siger"
```

Como esse projeto foi iniciado utilizando **Jupyter Notebook** (escolha pessoal para desenvolvimento do projeto), acabamos nos deparando com um erro por conta do arquivo não estar rodando em **.py** e sim em **.ipynb**. Então foi criado uma tentativa e uma exceção para que encontre o caminho do arquivo CSV:

```
try:
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
except NameError:
    base_dir = os.getcwd()

if os.path.basename(base_dir) == "src":
    base_dir = os.path.dirname(base_dir)
```

Então, se o código estiver rodando em arquivo .py, ele funcionaria perfeitamente por conta da primeira tentativa, se ele estiver rodando em .ipynb, a exceção será executada para poder funcionar as opções de menu do cenário dentro do Jupyter Notebook que será visto mais abaixo neste relatório.

Agora essa parte serve para localizar corretamente o arquivo **dados.csv**.

```
caminho_telemetria = os.path.join(base_dir, "data", "dados.csv")
```

4 Cenários Simulados

Antes de pedir para a inteligência artificial fazer o arquivo de telemetria (**dados.csv**), criamos 10 cenários simulados na mão, para depois pedir que esses cenários fossem convertidos em um arquivo **CSV**. A criação desses 10 cenários foi feito para que pudéssemos ter um menu, para que o usuário escolhesse e pudesse analisar diferentes cenários, ao invés de apenas 1. A partir desse pensamento, foi criado um dicionário chamado **cenarios** e com o valor da chave que o usuário digitaria, ele poderia acessar um cenário específico no terminal como por exemplo: **cenarios["1"]**.

Exemplo de como foi criado, antes de ser convertido em um arquivo .csv:

```
cenarios = {
    "1": criar_cenario(
        nome="Cenário Normal",
        descricao="Todos os sistemas principais estão funcionando dentro dos limites seguros.",
        classificacao_esperada="NORMAL",
        modulos={
            "suporte_vida": 1,
            "energia": 1,
            "comunicacao": 1,
            "habitat": 1,
            "laboratorio": 1,
            "armazenamento": 1,
            "navegacao": 1,
            "propulsao_auxiliar": 1
        },
        energia={
            "reserva_percentual": 86,
            "consumo_kwh": 42,
            "geracao_solar_kwh": 68,
            "geracao_auxiliar_kwh": 12
        },
        ambiente={
            "temperatura_interna": 23,
            "temperatura_externa": -63,
            "radiacao": "baixa",
            "qualidade_comunicacao": 92,
            "velocidade_vento": 18,
            "integridade_sensores": 98
        }
    )
}
```

```

},
historico_reserva=[88, 87, 86, 86, 85, 86],
historico_consumo=[40, 41, 42, 43, 42, 42],
historico_temperatura=[22, 23, 23, 24, 23, 23],
leituras=[
    ["00h", 65, 40, 88, 22, 95],
    ["04h", 64, 41, 87, 23, 94],
    ["08h", 68, 42, 86, 23, 93],
    ["12h", 70, 43, 86, 24, 92],
    ["16h", 66, 42, 85, 23, 92],
    ["20h", 68, 42, 86, 23, 92]
],
log_eventos=[
    "Inicialização do sistema concluída.",
    "Sensores calibrados.",
    "Comunicação estável com a base.",
    "Geração solar dentro do esperado.",
    "Reserva energética segura.",
    "Habitat em temperatura adequada.",
    "Laboratório operando normalmente.",
    "Nenhuma anomalia crítica detectada."
],
alertas_pendentes=[],
eventos_criticos=[],
inconsistencia_proposital=False,
recomendacao_inicial="Manter operação normal e continuar monitoramento."
),
"2": criar_cenario(
    nome="Cenário de Energia Baixa",
    descricao="A reserva de energia está baixa e o consumo está acima do ideal.",
    classificacao_esperada="ALERTA",
    modulos={
        "suporte_vida": 1,
        ...
    }
)

```

5 Leitura do Arquivo CSV

A função responsável por ler o arquivo CSV é:

```
ler_telemetria_csv(caminho_arquivo)
```

Ele abre o arquivo **dados.csv**, lê cada linha e transforma essa linha em um dicionário. Cada linha do arquivo CSV vai representar uma leitura da telemetria em determinado horário. Como temos 10 cenários e 6 leituras por cenário, o arquivo possui 60 linhas de dados.

6 Conversão dos Dados do Arquivo CSV

Quando o Python lê um arquivo CSV, os valores vêm como texto (string).

Por exemplo:

```
"88"
```

Mas o sistema precisa comparar valores numéricos, Por isso usamos a função:

```
converter_linha_csv(linha)
```

Essa função converte campos como energia, consumo, temperatura e status dos módulos para números inteiros (int).

Exemplo:

```
linha["reserva_percentual"] = int(linha["reserva_percentual"])
```

Isso é importante porque essa comparação só funciona corretamente com números:

Exemplo:

```
reserva < 40
```

Se **reserva** estivesse como texto (string), o sistema poderia gerar erro ou interpretar incorretamente.

7 Filtragem do Cenário Escolhido

O usuário escolhe um cenário no terminal e depois disso o sistema usa:

```
filtrar_telemetria_por_cenario(dados_telemetria, cenario_id)
```

Essa função vai percorrer todas as linhas do CSV e selecionar apenas as linhas do cenário escolhido.

Exemplo:

0 usuário escolheu o cenário 2 -> o Sistema filtra apenas as 6 linhas do cenário 2

8 Montagem do Cenário a Partir do CSV

Depois de filtrar as linhas do cenário, o sistema usa:

```
montar_cenario_a_partir_csv(linhas_cenario)
```

Essa função transforma as 6 linhas do CSV em uma estrutura organizada.

Ela cria:

- dicionário de módulos;
- dicionário de energia;
- dicionário de ambiente;
- listas históricas;
- matriz de leituras;
- fila de alertas;
- pilha de eventos críticos;
- log de eventos.

Essa função acaba por ser uma das mais importantes do projeto, porque ela vai transformando os dados tabulares do CSV em estruturas de dados que o Python consegue analisar com mais facilidade.

9 Estruturas de Dados Usadas

9.1 Lista

As listas armazenam séries de dados ao longo do tempo

Exemplos:

```
historico_reserva
historico_consumo
historico_temperatura
```

Essas listas permitem analisar tendências. Imagina que a reserva energética está caindo em várias leituras seguidas, o sistema pode prever risco futuro.

9.2 Dicionário

Os dicionários armazenam dados com chave e valor.

Exemplos:

```
modulos
energia
ambiente
cenario
```

Eles facilitam acesso direto aos dados.

Exemplo:

```
cenario["energia"]["reserva_percentual"]
```

9.3 Matriz

A matriz é representada por uma lista de listas.

Exemplo:

```
leituras = [
    ["00h", 65, 40, 88, 22, 95],
    ["04h", 64, 41, 87, 23, 94],
    ...
]
```

Cada linha representa um horário. Cada coluna representa uma variável, como geração, consumo, reserva, temperatura e comunicação

9.4 Fila

A fila aparece em: `alertas_pendentes` Ela representa alertas que precisam ser tratados.

O conceito de fila segue a ideia:

Primeiro que entra -> primeiro que sai

O famoso **FIFO** (First in, First out)

9.5 Pilha

A pilha aparece em: `eventos_criticos` - Ela registra eventos críticos recentes.

A pilha segue a ideia:

O último que entra -> primeiro que sai

O chamado **LIFO** (Last in, First Out)

9.6 Hierarquia

A hierarquia aparece em: `hierarquia_missao` - Ele representa sistemas e subsistemas da missão. Esse tipo de organização se aproxima de uma árvore.

10 Hierarquia da Missão

O código também possui uma estrutura chamada **hierarquia_missao**. Ela organiza a missão em sistemas e subsistemas, como energia, habitat e segurança.

Exemplo:

```
hierarquia_missao = {
  "energia": {
    "solar": "gera energia renovável",
    "baterias": "armazenam energia reserva",
    "consumo": "mede o gasto energético da missão"
  },
  "habitat": {
    "oxigenio": "mantém condições de sobrevivência",
    "temperatura": "controla o ambiente interno",
    "comunicacao": "mantém contato com a base"
  }
}
```

```
}  
}
```

Essa estrutura é parecida com uma árvore, porque existe um sistema principal dividido em partes menores.

11 Limites de Segurança

O sistema possui um dicionário chamado **limites_seguranca**.

Exemplo:

```
limites_seguranca = {  
    "energia_minima_alerta": 40,  
    "energia_minima_critica": 25,  
    "temperatura_interna_minima": 18,  
    "temperatura_interna_maxima": 28,  
    "temperatura_interna_critica": 34,  
    "qualidade_comunicacao_minima": 50,  
    "consumo_alto": 75  
}
```

Esse dicionário centraliza os limites usados nas regras de diagnóstico.

Por exemplo, em vez de escrever várias vezes o número 40 no código, usamos:

```
limites_seguranca["energia_minima_alerta"]
```

Isso deixa o sistema mais organizado e mais fácil de alterar. O dicionário permite acessar rapidamente um valor por meio de uma chave.

12 Exibição das Estruturas

Essa etapa organiza melhor os dados que já foram criados. Os dados não devem ficar soltos. Eles precisam ser organizados para que o sistema consiga analisar, buscar, comparar e tomar decisões. As funções a seguir têm o objetivo de mostrar como os dados foram organizados.

Exemplos:

```
exibir_tabela_status_modulos()  
exibir_vetor_energia()  
exibir_resumo_listas()  
exibir_resumo_matriz()  
exibir_fila_alertas_estruturada()  
exibir_pilha_eventos_criticos_estruturada()  
exibir_hierarquia_missao()
```

Essas funções não apenas mostram os dados. Elas ajudam a explicar os conceitos estudados. Por exemplo, a função `exibir_tabela_status_modulos()` transforma valores binários em informação legível:

1 -> NORMAL
0 -> FALHA

Isso conecta o projeto com lógica binária e lógica booleana.

13 Diagnóstico com Regras Lógicas

A função principal é `diagnosticar_missao(cenario)`, mas antes o sistema usa `avaliar_condicoes_logicas(cenario)` - Essa função transforma dados em variáveis booleanas.

Exemplo:

```
energia_baixa = reserva < 40  
energia_critica = reserva <= 25  
comunicacao_baixa = qualidade_comunicacao < 50  
radiacao_alta = radiacao == "alta"
```

Cada uma dessas variáveis podem ser True ou False.

14 Operadores Lógicos

O projeto utiliza os operadores lógicos para ajudar nas decisões.

14.1 Exemplo com AND

```
energia_baixa and consumo_maior_que_geracao
```

Essa regra só será verdadeira se as duas condições forem verdadeiras.

14.2 Exemplo com OR

```
falha_comunicacao_modulo or comunicacao_baixa
```

Essa regra será verdadeira se pelo menos uma das condições ocorrer.

14.3 Exemplo com NOT

```
falha_suporte_vida = not suporte_vida_funcionando
```

Se o suporte à vida não está funcionando, então existe falha no suporte à vida.

15 Classificação da Missão

Com o diagnóstico da missão, essa função classificará como NORMAL, ALERTA ou CRÍTICO.

A ordem de decisão:

1 - Primeiro verifica se há situação CRÍTICA.

- 2 - Depois verifica se há situação de ALERTA.
- 3 - Se nenhuma condição perigosa existir, classifica como NORMAL.

Essa ordem específica evita que uma falha grave seja ignorada.

Exemplo:

```
if critico:
    status = "CRÍTICO"
elif alerta:
    status = "ALERTA"
else:
    status = "NORMAL"
```

16 Expressão Booleana Principal

```
critico =
    falha_suporte_vida
    or energia_critica
    or temperatura_critica
    or (energia_baixa and consumo_maior_que_geracao)
    or (comunicacao_falha and radiacao_alta)
    or duas_ou_mais_falhas_essenciais
```

Essa expressão resume a lógica central do sistema. Mostra que o sistema considera crítico qualquer cenário em que a vida, a energia, a temperatura ou a combinação de falhas comprometa a segurança da missão.

17 Motivos e Recomendações

O sistema não só calcula um status como também explica o motivo. Isso é feito pela função: `gerar_motivos_diagnostico(condicoes)`.

Exemplo de motivo:

Reserva energética em nível crítico.

Depois, o sistema gera recomendações com:

```
gerar_recomendacoes_diagnostico(condicoes, status)
```

Exemplo de recomendação:

Desligar sistemas não essenciais e preservar energia para suporte à vida e habitat.

Isso é importante porque um sistema inteligente precisa comunicar suas decisões de forma compreensível.

18 Previsão de Dados

A variável escolhida para previsão de dados foi: `historico_reserva` - Essa lista representa a reserva energética ao longo dos ciclos de leitura.

Exemplo:

```
[52, 47, 43, 39, 36, 34]
```

O sistema usa regressão linear simples para prever a reserva do próximo ciclo.

A função principal é:

```
prever_reserva_energia(cenario)
```

Ela chama outras funções menores:

```
calcular_media()
calcular_regressao_linear_simples()
interpretar_tendencia()
gerar_recomendacao_preditiva()
```

19 Regressão Linear Simples

A regressão linear tenta ajustar uma reta aos dados - A forma geral da reta é: $y = a * x + b$

No projeto:

- `x` representa o ciclo da leitura;
- `y` representa a reservar energética;
- `a` representa a inclinação da reta;
- `b` representa o intercepto.

Se a inclinação for negativa, a tendência é de queda. Se for positiva, a tendência é de aumento. Se estiver próxima de zero, a tendência é de estabilidade.

20 Previsão Bruta e Previsão Ajustada

A previsão bruta é o valor calculado diretamente pela regressão.

Exemplo:

```
previsao_bruta = inclinacao * proximo_ciclo + intercepto
```

Já a previsão ajustada é a previsão corrigida para respeitar o limite de porcentagem. Como estamos falando de reserva energética, o valor precisa ficar entre 0% e 100%

Exemplo:

```
if previsao_bruta < 0:
    previsao_ajustada = 0
elif previsao_bruta > 100:
```

```

    previsao_ajustada = 100
else:
    previsao_ajustada = previsao_bruta

```

Portanto, como não temos valores menor que 0 ou maiores que 100 nos cenários, eles sempre serão iguais. Mas caso tenhamos um valor que ultrapasse esses valores, o sistema ajustará. Por exemplo: -5% ou 103%, o sistema ajustará para 0% ou 100%.

21 Recomendação Preditiva

A previsão além de ser exibida, ela também influencia através de recomendações.

A função:

```

gerar_recomendacao_preditiva(previsao, tendencia)

```

Usa a previsão para decidir o que recomendar.

Portanto:

```

if previsao <= 25:
    return "CRÍTICO"
elif previsao < 40:
    return "ALERTA"
elif tendencia == "queda":
    return "ATENÇÃO"
else:
    return "NORMAL"

```

Com isso o sistema evolui de um comportamento apenas reativo para um comportamento preditivo.

22 Menu Principal e Loop do Sistema

A função principal do programa é o **main()**. Essa função mantém o sistema ativo utilizando **while True:**. Esse laço faz com que o programa continue rodando até o usuário digitar **sair**. Caso o usuário digite uma opção errada, o sistema usará o **try** e o **except** para tratar o erro sem quebrar o programa.

Exemplo:

```

try:
    ...
except ValueError as erro:
    print(f"Erro: {erro}")
except Exception as erro:
    print(f"Ocorreu um erro inesperado. {erro}")

```

O **Exception** irá capturar os erros mais gerais, já o **ValueError** irá capturar erros específicos. Todo **ValueError** é um **Exception**, mas nem todo **Exception** é um **ValueError**.

```

except ValueError as erro:

```

Significa: "Trate aqui erros causados por valores inválidos."

Exemplo:

```
numero = int("abc")
```

Isso gera um `ValueError`, porque "abc" não pode ser convertido para número inteiro.

```
except Exception as erro:
```

Significa: "Trate aqui qualquer erro comum que ainda não foi tratado antes."

Exemplo:

```
lista = [10, 20]
```

```
print(lista[5])
```

Isso não é um `ValueError`. É um `IndexError`, porque a posição 5 não existe na lista. Mas `Exception`

`ValueError` serve para errors esperados, como lista vazia, cenário inexistente ou dados insuficientes.

`Exception` serve como uma proteção final para erros inesperados, como erro de digitação no código, chave inexistente em dicionário, variável não definida, índice errado, etc.

Por isso o **`ValueError`** vêm antes do **`Exception`**, porque se o **`Exception`** for colocando primeiro, ele captura tudo antes, e o **`ValueError`** nunca será alcançado.

23 Conclusão

Primeiro o projeto definiu os cenários e os dados da missão. Depois, organizou esses dados em estruturas computacionais. Em seguida, passou a ler os dados a partir de um arquivo CSV, tornando o sistema mais realista. Depois adicionou regras lógicas para diagnóstico e, por fim, incluiu previsão de dados para apoiar decisões preventivas.

O sistema não é apenas um programa que imprime valores. Ele simula uma central de monitoramento capaz de receber dados, organizar informações, detectar riscos, gerar diagnósticos, explicar motivos e recomendar ações.